



中国科学院大学
University of Chinese Academy of Sciences

Deep Learning

Deep Generation Network

Xinfeng Zhang (张新峰)

School of Computer Science and Technology

University of Chinese Academy of Sciences

Email: xfzhang@ucas.ac.cn



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY

决策树

□ 决策树(Decision Tree)

- 是一种基于树结构进行决策的机器学习方法，这恰是人类面临决策时一种很自然的处理机制

□ 决策树生成过程：

- 寻找最适合分割的特征
- 根据纯度判断方法，寻找最优的分割点，基于这一特征把数据分割成纯度更高的两部分数据
- 判断是否达到要求，若未达到，重复步骤一继续分割，直到达到要求停止为止
- 剪枝，防止过拟合

随机森林

□ 随机森林 (Random Forest)

- 用随机的方式建立起一棵棵决策树，然后由这些决策树组成一个森林，其中每棵决策树之间没有关联，当有一个新的样本输入时，就让每棵树独立的做出判断，按照**多数原则**决定该样本的分类结果。这是一种典型的**集成学习**思想
- **集成学习 (Ensemble learning)**
 - 组合多个弱监督模型以期得到一个更好更全面的强监督模型，集成学习潜在的思想是即便某一个弱分类器得到了错误的预测，其他的弱分类器也可以将错误纠正回来

随机森林

□ 随机森林的生成方法

- 从样本集中通过重采样的方式产生 n 个样本
- 假设样本特征数目为 a ，对 n 个样本选择 a 中的 k 个特征，用建立决策树的方式获得最佳分割点(例如ID3算法)
- 重复 m 次，产生 m 棵决策树

□ 预测步骤：多数投票机制来进行预测

- 向建立好的随机森林中输入一个新样本
- 随机森林中的每棵决策树都独立的做出判断
- 将得到票数最多的分类结果作为该样本最终的类别

深度森林

- 神经网络可以堆叠为深度神经网络，并且取得了显著的效果。那我们可以考虑，是不是可以将其他的学习模型堆叠起来，以获取更好的表示性能，深度森林模型就是基于这种想法提出来的一种深度结构
- 2017年，深度森林由周志华老师提出，基于**树模型**的方法，主要使用**集成学习**思想方法的深度学习框架，也可作为一种在某些任务下替代深度神经网络的方法

- [Deep Forest: Towards an Alternative to Deep Neural Networks](#), 2017

深度森林

□ 深度神经网络的问题

- 需要使用大量的数据进行训练
- 计算资源消耗大
- 可解释性差
- 超参数多，训练依赖于经验

□ 深度森林的优势

- 与DNN相比需要的参数更少
- 训练速度快
- 不仅适合大规模数据也适合小规模数据
- 基于树模型解释性比较好

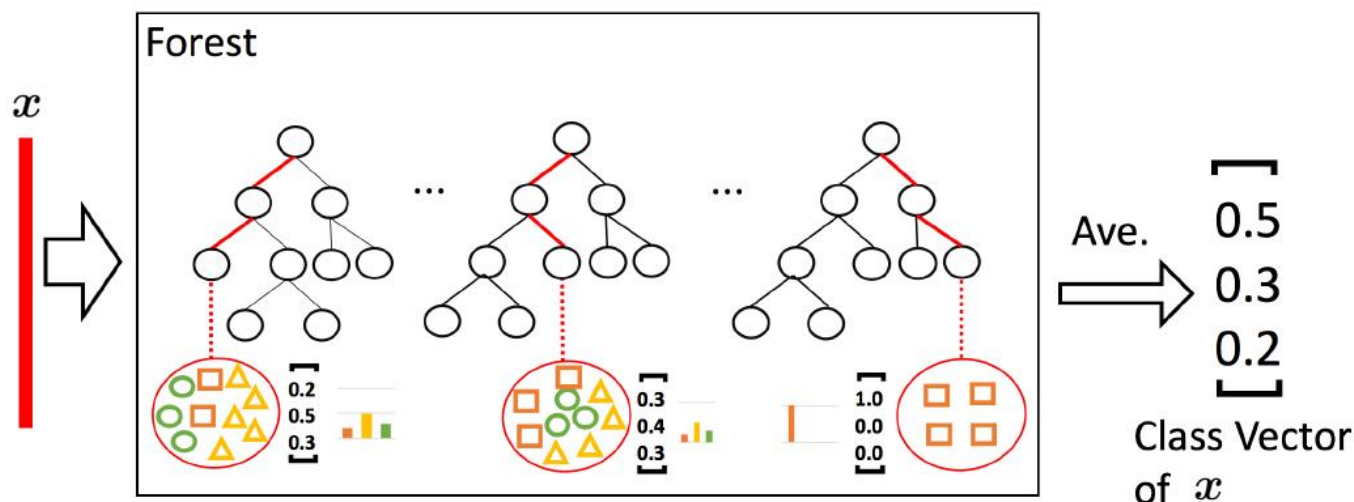
多粒度级联森林

- ❑ 每个完全随机的森林包含完全随机树，通过随机选择一个特征在树的每个节点进行分割实现生长，直到叶子节点只包含相同类的实例。
- ❑ 类似的，每个随机森林包含的树，通过随机选择 $\sqrt{d}$ 数量的特征作为候选（ d 是输入特征的数量），然后选择具有最佳gini值的特征作为分割
- ❑ 每个森林中的树的数值是一个超参数

多粒度级联森林

□ GCForest每个森林的类分布向量生成流程：

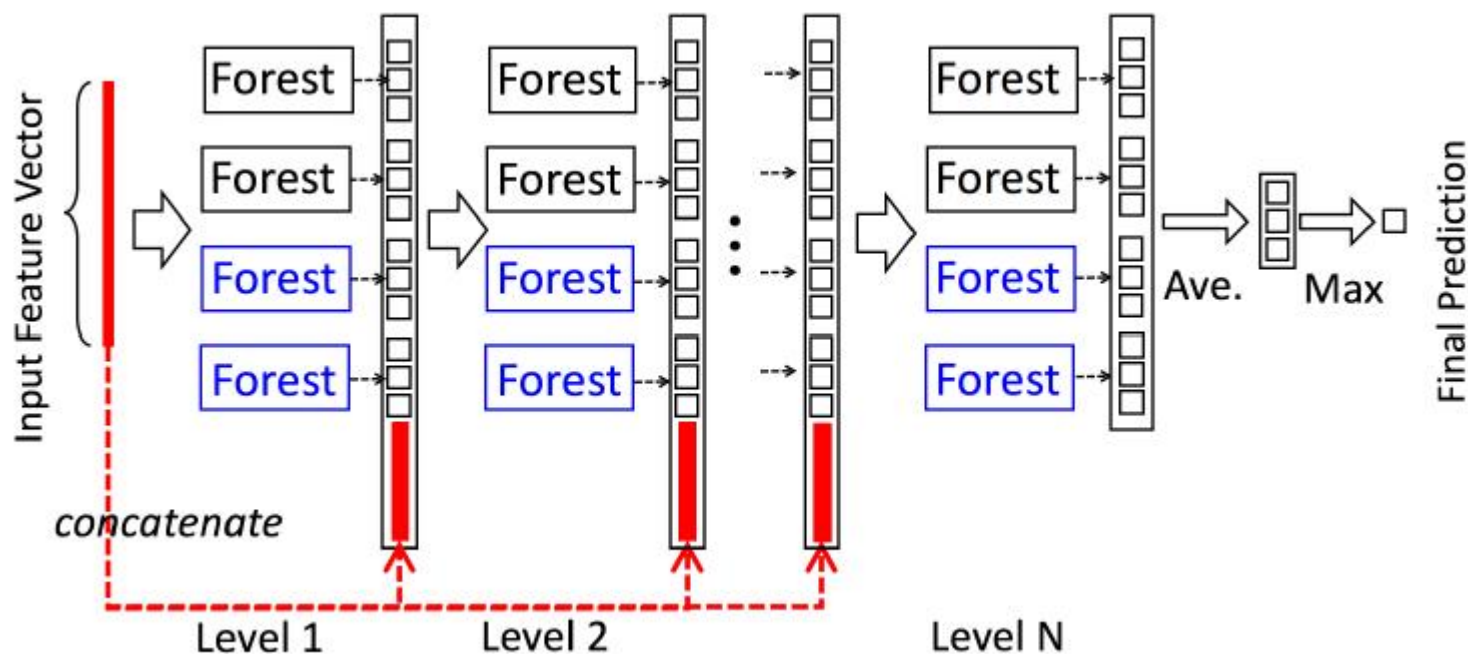
- 对于测试样本，每棵树会根据样本所在的子空间中训练样本的类别占比生成一个类别的概率分布，然后对森林内所有树的各类比例取平均，输出整个森林对各类的比例
- 每个森林都会生成长度为C的概率向量，如果一层有N个森林，那么每个森林生成的C个元素会拼接在一起，组成C*N个元素向量，这就是一层的输出



多粒度级联森林

□ GCForest每个森林的类分布向量生成流程：

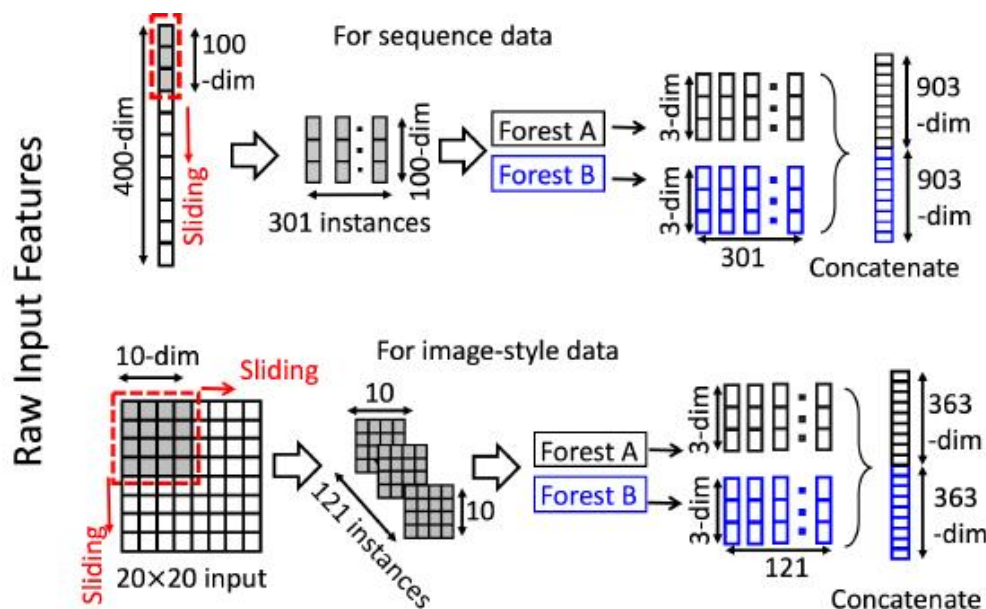
- 每个森林都会生成长度为C的概率向量，如果一层有N个森林，那么每个森林生成的C个元素会拼接在一起，组成C*N个元素向量，这就是一层的输出
- 这基础上把源输入特征向量拼接上去组成了下一层的输入



多粒度级联森林

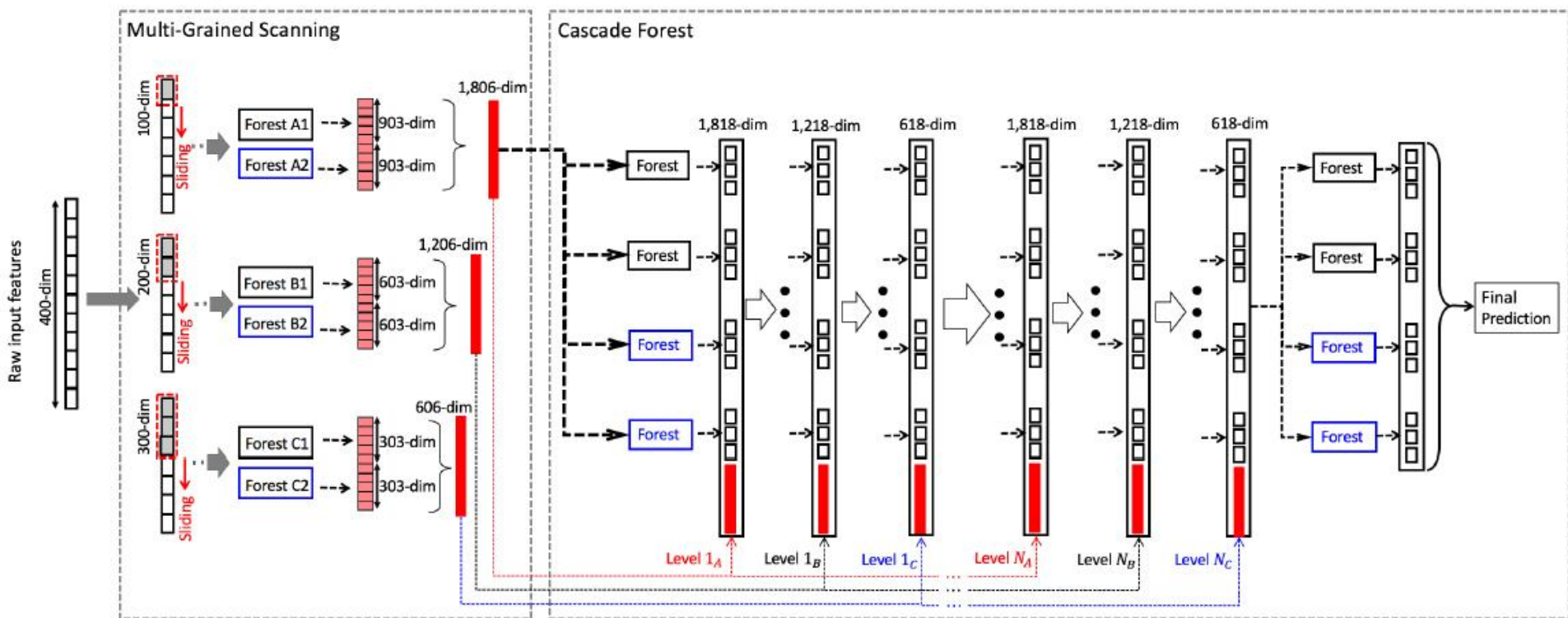
□ GCForest的多粒度扫描:

- 采用滑动窗口的方法，首先生成若干个实例，其次通过实例生成两个森林，一个完全随机森林，一个随机森林，接着再把生成的两个森林生成对应的相同维度的“类向量”，最后把这两大类向量连接在一起。
- 一般来说会采用多个不同大小的窗口做扫描



多粒度级联森林

□ 整体结构



多粒度级联森林

□ 超参数对比

Deep neural networks (e.g., convolutional neural networks)	gcForest
Type of activation functions: Sigmoid, ReLU, tanh, linear, etc. Architecture configurations: No. Hidden layers: ? No. Nodes in hidden layer: ? No. Feature maps: ? Kernel size: ? Optimization configurations: Learning rate: ? Dropout: {0.25/0.50} Momentum: ? L1/L2 weight regularization penalty: ? Weight initialization: Uniform, glorot_normal, glorot_uni, etc. Batch size: {32/64/128}	Type of forests: Completely-random tree forest, random forest, etc. Forest in multi-grained scanning: No. Forests: {2} No. Trees in each forest: {500} Tree growth: till pure leaf, or reach depth 100 Sliding window size: $\{\lfloor d/16 \rfloor, \lfloor d/8 \rfloor, \lfloor d/4 \rfloor\}$ Forest in cascade: No. Forests: {8} No. Trees in each forest: {500} Tree growth: till pure leaf

多粒度级联森林

□ 相关性能

– 手写数字识别(MNIST)

gcForest	99.26%
LeNet-5	99.05%
Deep Belief Net	98.75% [Hinton <i>et al.</i> , 2006]
SVM (rbf kernel)	98.60%
Random Forest	96.80%

– 人脸识别(ORL)

	5 image	7 images	9 images
gcForest	91.00%	96.67%	97.50%
Random Forest	91.00%	93.33%	95.00%
CNN	86.50%	91.67%	95.00%
SVM (rbf kernel)	80.50%	82.50%	85.00%
kNN	76.00%	83.33%	92.50%

– 音乐分类(GTZAN)

gcForest	65.67%
CNN	59.20%
MLP	58.00%
Random Forest	50.33%
Logistic Regression	50.00%
SVM (rbf kernel)	18.33%

– 情感分类(IMDB)

gcForest	89.16%
CNN	89.02% [Kim, 2014]
MLP	88.04%
Logistic Regression	88.62%
SVM (linear kernel)	87.56%
Random Forest	85.32%

基于森林的自编码器

□ 自编码器 (Auto-Encoder)

- 是神经网络的一种，是一种重要的表示学习模型，是深度学习的关键要素之一。自编码器的基本结构是由一个编码器 (encoder) 和一个解码器 (decoder) 组成，其中 encoder 将输入映射到隐空间，decoder 将隐空间的表示重构为原表示

□ eForest (Encoder-Forest)

- 基于森林的自编码器，能够利用决策树的决策路径所定义的等效类来进行后向重建。利用决策树集成算法进行向前编码和向后解码的操作
- [AutoEncoder by Forest](#), AAAI 2017

基于森林的自编码器

□ 前向编码

- 在一个有 N 个已训练决策树的森林中，前向编码过程接受输入数据并将其发送到集成方法中每棵树的根结点
- 一旦数据遍历 (traverse) 到所有树的叶结点，该过程将返回 T 维向量， T 中的每个元素 t 是树 t 中的叶结点的整数索引

基于森林的自编码器

□ 前向编码

Algorithm 1: Forward Encoding

Input: A trained forest F with T trees,
An input data x

Output: x_{enc}

$x_{enc} \leftarrow \text{zeros}[T,1]$ % initialize x_{enc}

for i *in* $\text{range}(T)$ **do**

$x_{enc}[i] \leftarrow F.\text{tree}[i].\text{get_leaf_index}(x)$

end

return x_{enc}

基于森林的自编码器

□ 后向解码

- 利用决策树进行决策时需要将决策的路径记录下来，而每个决策路径对应了一个规则（rule）

识别出来的路径用红色突出显示，每个路径对应一个符号规则。

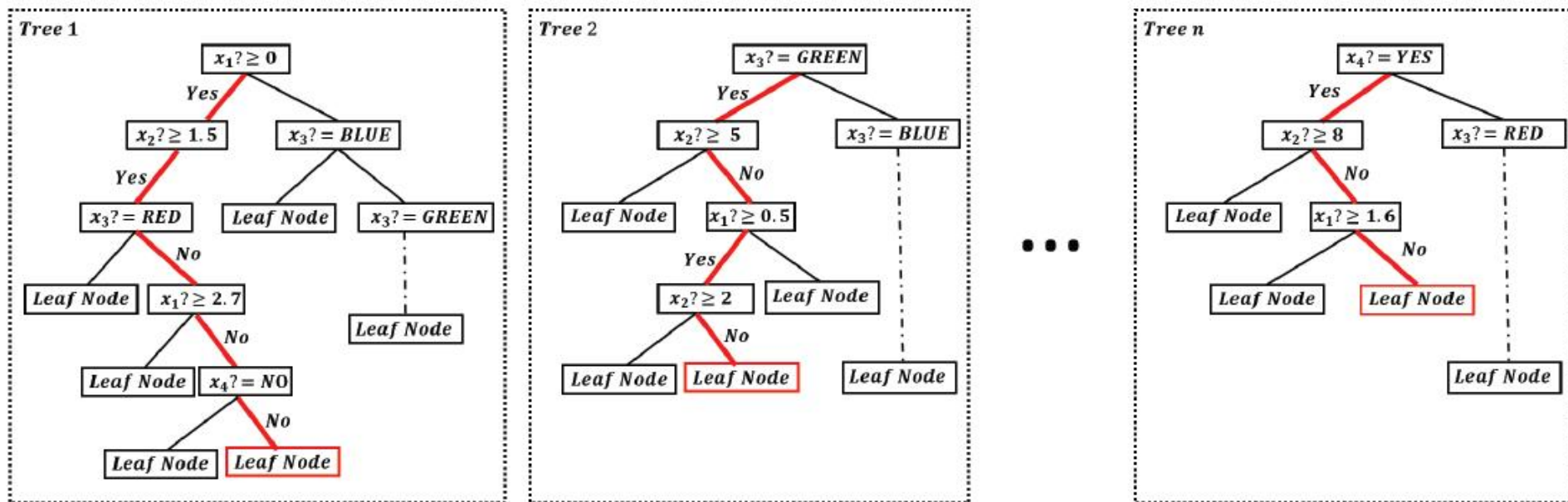


Figure 1: Traversing backward along decision paths

基于森林的自编码器

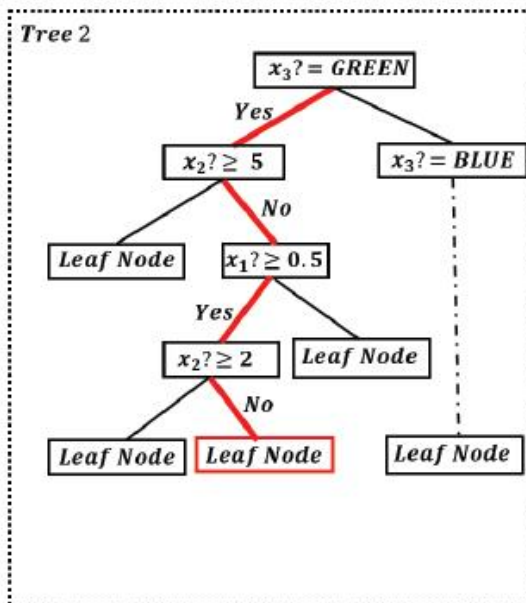
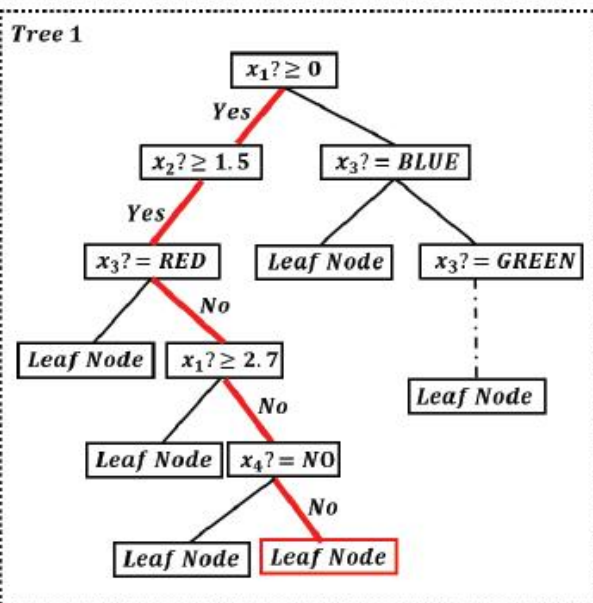
□ 后向解码

- 利用决策树进行决策时需要将决策的路径记录下来，而每个决策路径对应了一个规则 (rule)

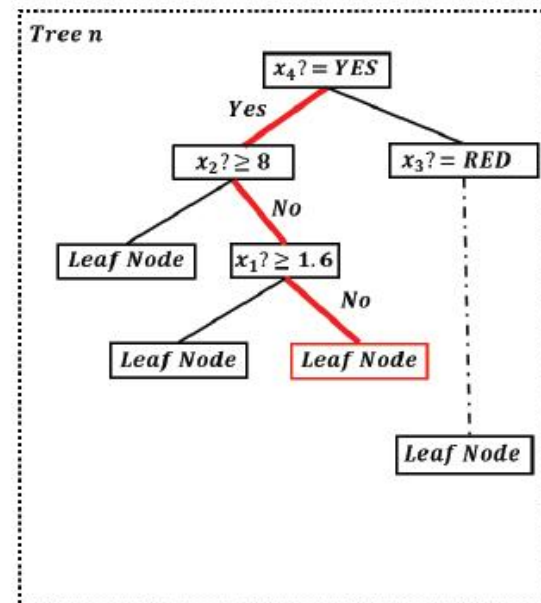
$$RULE_1: (x_1 \geq 0) \wedge (x_2 \geq 1.5) \wedge \neg(x_3 == RED) \wedge \neg(x_1 \geq 2.7) \wedge \neg(x_4 == NO)$$

$$RULE_2: (x_3 == GREEN) \wedge \neg(x_2 \geq 5) \wedge (x_1 \geq 0.5) \wedge \neg(x_2 \geq 2)$$

$$RULE_n: (x_4 == YES) \wedge \neg(x_2 \geq 8) \wedge \neg(x_1 \geq 1.6)$$



...



基于森林的自编码器

□ 后向解码

- 利用决策树进行决策时需要将决策的路径记录下来，而每个决策路径对应了一个规则 (rule)

$$RULE_1: (x_1 \geq 0) \wedge (x_2 \geq 1.5) \wedge \neg(x_3 == RED) \wedge \neg(x_1 \geq 2.7) \wedge \neg(x_4 == NO)$$

$$RULE_2: (x_3 == GREEN) \wedge \neg(x_2 \geq 5) \wedge (x_1 \geq 0.5) \wedge \neg(x_2 \geq 2)$$

...

$$RULE_n: (x_4 == YES) \wedge \neg(x_2 \geq 8) \wedge \neg(x_1 \geq 1.6)$$



$$RULE'_1: (2.7 > x_1 \geq 0) \wedge (x_2 \geq 1.5) \wedge \neg(x_3 == RED) \wedge (x_4 == YES)$$

$$RULE'_2: (x_1 \geq 0.5) \wedge \neg(x_2 \geq 2) \wedge (x_3 == GREEN)$$

...

$$RULE'_n: \neg(x_1 \geq 1.6) \wedge \neg(x_2 \geq 8) \wedge (x_4 == YES)$$

基于森林的自编码器

□ 后向解码

- 利用决策树进行决策时需要将决策的路径记录下来，而每个决策路径对应了一个规则（rule）
- 在进行解码重构的过程中，利用森林中N个规则，运用最大相容规则Maximum Compatible Rule（MCR），获取N维编码所对应的更精确的规则，以此规则进行重构

$$RULE'_1 : (2.7 > x_1 \geq 0) \wedge (x_2 \geq 1.5) \wedge \neg(x_3 == RED) \wedge (x_4 == YES)$$

$$RULE'_2 : (x_1 \geq 0.5) \wedge \neg(x_2 \geq 2) \wedge (x_3 == GREEN)$$

...

$$RULE'_n : \neg(x_1 \geq 1.6) \wedge \neg(x_2 \geq 8) \wedge (x_4 == YES)$$



$$(1.6 > x_1 \geq 0.5) \wedge (2 > x_2 \geq 1.5) \wedge (x_3 == GREEN) \wedge (x_4 == YES)$$

基于森林的自编码器

□ 后向解码—计算出最大相容规则MCR

Algorithm 2: Calculate MCR

Input: A path rule list *Rule_List* consists of T rules defined by a forest with T trees

Output: *MCR*

$MCR \leftarrow initialize_list()$

for i in range(T) **do**

$path_rule \leftarrow rule_list[i]$

for $node_rule$ in $path_rule.node_rule_list$ **do**

$j \leftarrow node_rule.get_attribute()$

$bound \leftarrow node_rule.get_bound()$

$MCR[j] \leftarrow intersect(MCR[j], bound)$

end

end

return *MCR*

基于森林的自编码器

□ 后向解码

- 对于类别属性，例如 x_3 和 x_4 原始样本就取MCR中的值
- 对于数值属性，例如 x_2 ，可以去区间内的代表值
- 那么重构的样本 $x=[0.5,1.5, \text{GREEN}, \text{YES}]$

Algorithm 3: Backward Decoding

Input: x_{enc} , trained eForest F with T trees

Output: x_{dec}

$rule_list \leftarrow initialize_list()$

for i **in** $range(T)$ **do**

$path \leftarrow F.tree[i].get_path(x_{enc}[i])$

$path_rule \leftarrow calculate_rules(path)$

$path_rule \leftarrow simplify(path_rule)$

$rule_list.append(path_rule)$

end

$MCR \leftarrow calculate_MCR(rule_list)$

$x_{dec} \leftarrow sample(MCR, method='minimum')$

return x_{dec}

基于森林的自编码器

□ 优势

- 训练速度较快：如在MNIST和 CIFAR10 上的训练速度比基于神经网络的模型快数倍以上
- 重构误差低：基于规则，而不是计算
- 容损性：编码规则具有强相关性，因此在部分损坏的情况下也能很好地工作
- 可复用性：在一个数据集上训练好的模型能够直接应用于同领域另外一个数据集

□ 不足

- 与神经网络相比编码表达力不足
- 编码、解码速度相对神经网络较慢

深度森林的适用条件

- 具备逐层处理的任务
- 参数不可微
- 一定的模型复杂度

谢谢！



计算机科学与技术学院

SCHOOL OF COMPUTER SCIENCE AND TECHNOLOGY